Managing Unexpected NAND Flash Power Loss in Embedded Systems

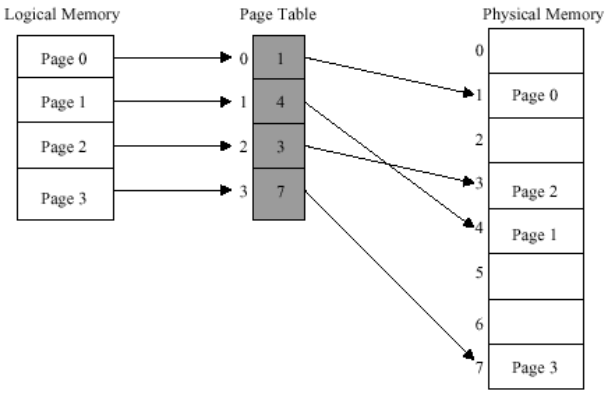
1. Introduction

SLC NAND flash is now becoming the main system storage for executable code and data in embedded systems. It is imperative that the system storage design be reliable and be tolerant of any system failures. When choosing NAND flash as the main system storage, many reliability issues must be considered such as ECC coverage for possible data error bits, block wear leveling to extend the whole system's longevity, and the ability to recover from unexpected power loss during system operation--especially when storing code and data. This article will focus on handling the power loss issue from a software point of view. It will start by presenting an outline of an ordinary flash management algorithm and show how it manages the whole flash during normal operations and when a power loss occurs. It will examine the possible negative consequences of an unexpected power loss and discuss a recovery technique to return to normal operation.

2. The Flash Management Algorithm Outline

Due to the characteristics of NAND flash, flash management methods must be implemented to help upper layer software utilize NAND flash as a reliable storage device. The first method is called the FTL (Flash Translation Layer), which is basically a logical to physical address mapping scheme. Figure 2-1 shows a Flash Translation Layer Structure. When a request for reading data from a logical address is received, the FTL very efficiently locates the correct physical address and reads the data.

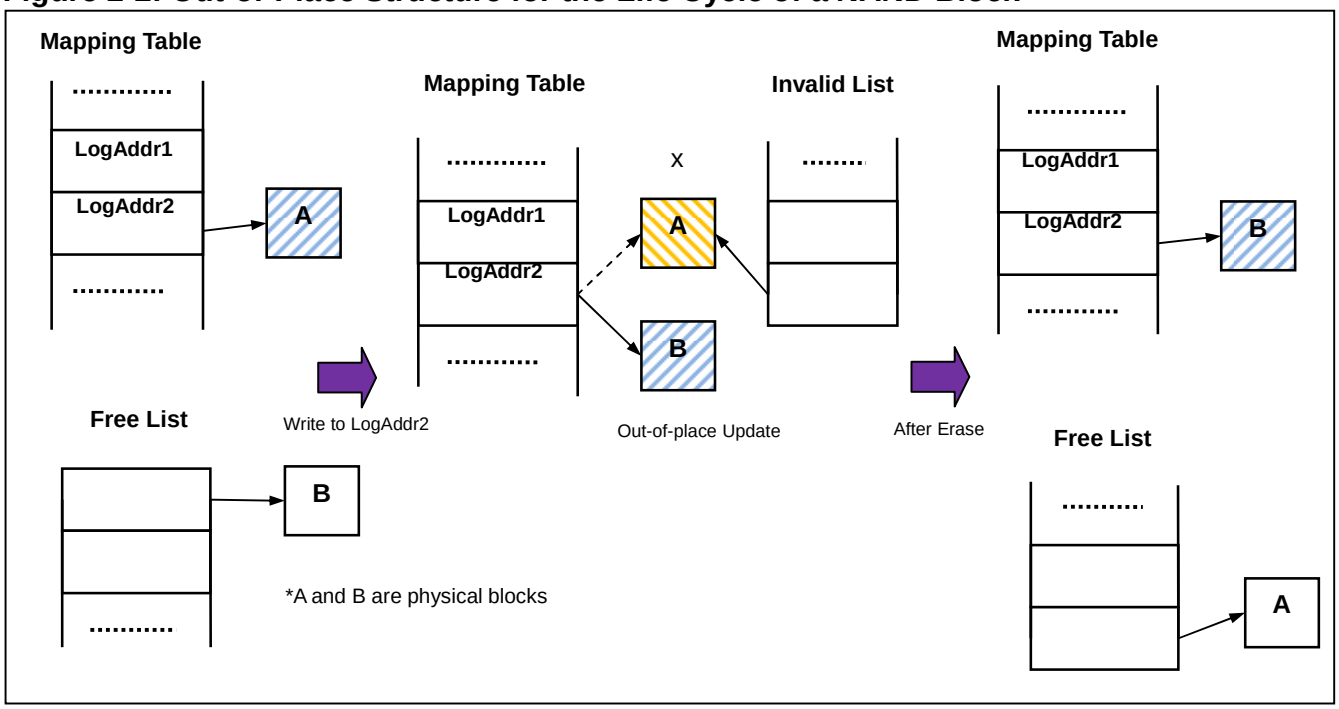
Figure 2-1: Flash Translation Layer Structure



Notice that due to the existence of bad blocks, the use of an *out-of-place update* feature, and system performance considerations, the logical address space is actually slightly less than the real physical address space available in the NAND flash. The extra physical address space is used to reserve empty free blocks, which can be immediately used for new write requests without having to erase a previously used block. After new data is written to the free space, the logical

to physical mapping is updated accordingly. The old space is marked dirty (invalid) and waits for an idle period where it can be erased and returned to the free space pool. The life cycle of a NAND block is depicted in Figure 2-2. It shows how data stored at logical address 2 which is stored in physical block A is updated in physical block B. Logical address 2 is then updated to point to physical block B, and physical block A can then be erased and returned to the pool of free (available) blocks for later use.

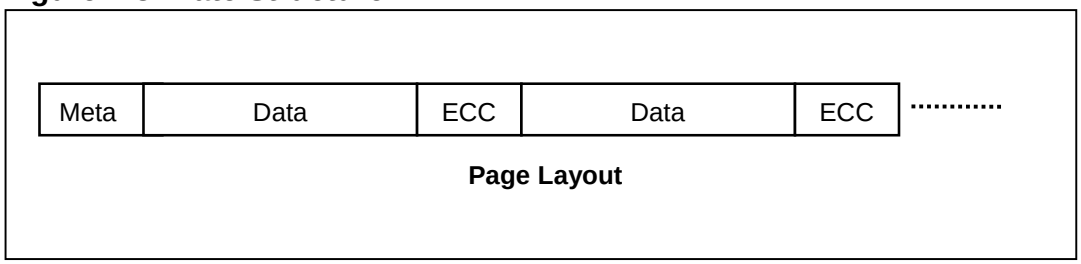
Figure 2-2: Out-of-Place Structure for the Life Cycle of a NAND Block



We can see that flash management software must maintain a Logical to Physical *Mapping Table* and *Free* and *Invalid Lists* in order to support efficient and reliable read and write operations.

Since the 3 data structures (Mapping Table, Free List, and Invalid List) are typically stored in volatile memory, how can we reconstruct them during the next system power on cycle? The solution is to store *Meta Data* along with the Real Data as shown in the Figure 2-3 Data Structure. Note that the actual layout maybe different and will depend on the system core chip (SoC) used. Meta data can be at the beginning or the end of the data sectors. The following page layout is used by the GPMI NAND flash controller of the FreeScale iMX28 SoC.

Figure 2-3: Date Structure



So what is meta data all about? In our example, It contains all of the information necessary to reconstruct the mapping table and the free and invalid lists. Figure 2-4 is an example of a meta data structure:

Figure 2-4: Meta Data Structure

```
MetaData:
     UINT MetaMagic;
     UINT EraseCount;
     USHORT LogicalAddress;
     USHORT LastPhyNum;
     UCHAR Status;
```

Each field is explained as follows:

- The *MetaMagic* field is a specific random number (ex. 0x0C00FFEE) that is used to identify the meta data as real and valid.
- The *EraseCount* keeps track of the number of times a block has been erased since the flash device was first put into service. This is an important piece of information need by the system to manage *Wear-Leveling*. Note that the maximum number of erase cycles is different for SLC(100K) and MLC (5K) NAND flash, so a two-byte integer is enough for MLC. Consult your device datasheet to confirm program/erase cycling limits for your flash.
- The *LogicalAddress* field is the key information to reconstruct the mapping table. While writing the data that shifts the logical mapping from physical block A to B, there could be moments where a logical address points to two different physical blocks. This will be resolved by the LastPhyNum field.
- The LastPhyNum is used to keep track of the mapping change record (i.e. in B's *LastPhyNum,* it points to A), so that if two physical blocks both have the same Logical address, the old one should be considered invalid if the new one is successfully written. Note that this field is also helpful in the later power loss discussions.
- The Status field is used to indicate whether the block is a regular one, if it is for some system specific purpose that should not be included in the logical mapping table, if the data in the block is sequentially or randomly written, or if it is for other purposes requiring special treatment. Notice that the sequences of these fields are arranged intentionally. The MetaMagic and EraseCount will be the first 2 fields written to a flash page. This is

helpful if the block is already erased but not assigned to any specific logical address yet. The flash management software can write these 2 fields first to keep the erase count of the block and indicate that the block is a valid free one.

So every time the system is powered on, the flash management software should scan the meta data structure (not the whole page) of each physical block to reconstruct the mapping table and the free and invalid lists. For every block that has valid meta data, update the mapping entry with the *LogicalAddress* index pointing to the physical block address. If the entry is already occupied by another physical block, then check the LastPhyNum field to decide which one is the latest valid one, and add the other one to the *Invalid List* for later erasing. If the block is empty (all 0xFF) or the meta data only contains the magic code and erase count (the rest are all 0xFF), this block should go to the *Free List*. After scanning all of the blocks, the reconstruction is now completed. A conceptual software code for scanning the whole flash may look like the following:

```
LogToPhy[0 .. N] = -1;    // Initialize all the mapping entries to invalid
For each PhyBlock[i] do
   ReadFlash(PhyBlock[i].Meta);    // read the meta data only
   If (PhyBlock[i].Meta.Magic == VALID_MAGIC_CODE) {
        UINT theLogicaAddr = PhyBlock[i].Meta.LogicalAddress;
        If (theLogicalAddress == 0xFFFFFFFF)
             Append(FreeList, PhyBlock[i]);
        Else If (LogToPhy[theLogicalAddress] == -1)       // not occupied yet
             LogToPhy[theLogicalAddress] = i;
        Else {
             If (PhyBLock[i].Meta.LastPhyNum == LogToPhy[theLogicalAddr]) {
                  Append(InvalidList, PhyBlock[LogToPhy[theLogicalAddr]]);
                  LogToPhy[theLogicalAddress] = i;
             } Else
                  Append(InvalidList, PhyBlock[i]);
        }
    }
     Else     // Invalid Meta.Magic code
        Append(InvalidList, PhyBlock[i]);
End loop
```

3. When Will an Unexpected Power Loss Become an Issue?

Flash management software will perform as described in previous sections if nothing strange happens. But if power is suddenly lost, accidentally or intentionally, when a reading or writing process is ongoing, we must consider the possibility that data corruption might have occurred and how to recover if it did.

If power is lost in the moment of reading, the system will be ok since no data will be altered, but if power is lost while the flash is busy with a program or erase operation, data in flash cells targeted by the program or erase operation may be corrupted. Below we list the possible power loss scenarios and recommend which special recovery method should be considered:

- The page (sector) data along with all the meta data and ECC are all transferred and ready at the flash side and the programming process begins but not finished yet.
- The block is currently being erased and has not yet finished.

Both of the above are instances where the *busy* pin of the NAND flash remains low during programming or erasing, and does not return high to indicate process completion.

In the first case, the data looks fine at the first few pages, those pages that are successfully written prior to the power loss, but after the good ones, there would be a page (or sector) whose data is corrupted, and after that, the remaining pages should be all empty (0xFF).

In the second case, data in the entire block is suspect. Some fast cells may have reached the erased state (“1”), but slower cells may have stopped somewhere in between the ”0” and “1” state.

4. System Software Solutions for NAND Power Loss

The solutions to these power loss issues (either during programming or block erase) are first to identify which is the *Unfinished Programed Page* or *Unfinished Erased block*. The *unfinished programed page* has a few symptoms that can be easily identified:

- ECC Errors when reading the page (either correctable or uncorrectable), or the ECC codes are all 0xFF which is not likely for a valid ECC.
- If the page size is 2KB or more, there are remaining blank sectors (all 0xFF) in a partially written page. This will not occur in normal flash management algorithms.
- Two different physical blocks are mapped to the same logical address, but the new one has unfinished (empty) pages at the rear end of the block. This situation is usually detected during the meta data scan when the system is powered on.

If any of the above situations occurs, some recovery actions should be taken, for example, move the still valid data to a new block, then erase the old one.

The *Unfinished Erased Block* case is more difficult to identify because it may just looks like a fresh new block if you don't scan the whole block carefully. But if the flash management algorithm writes something to the block immediately after erasing (such as in the Meta Data description), it will provide a way to identify if it is an *Unfinished Erased Block* or not (no meta data = unfinished erased block).

Another method is to maintain an erasing journal in a hidden block (like all journaling flash file systems, e.g. JFFS and JFFS2), that writes something like "Before erase block 0xXXXX …" in a sector before the erasing process. After the erase had completed successfully, append another sector like "Erase block 0xXXXX done". So whenever the system restarts again, it can check the journal to see if the erasing of the block is complete or not. If an *Unfinished Erased Block* is found, the block should be erased again. This method can also be used to keep the programming records for the same purpose. But be aware that the wear leveling issue of the hidden block should be considered carefully because this will be a heavily used block.

Although we have not covered all the power loss cases and the proposed solutions may not be sufficient, taking these solutions into consideration is a good first step necessary to create a more robust system that can handle power failure issues. Different platforms may have different approaches to this issue. These solutions may also degrade the system performance to some degree.

5. Revision History

Revision	Description	Date
1.0	Initial Release	Dec. 17, 2014

Except for customized products which have been expressly identified in the applicable agreement, Macronix's products are designed, developed, and/or manufactured for ordinary business, industrial, personal, and/or household applications only, and not for use in any applications which may, directly or indirectly, cause death, personal injury, or severe property damages. In the event Macronix products are used in contradicted to their target usage above, the buyer shall take any and all actions to ensure said Macronix's product qualified for its actual use in accordance with the applicable laws and regulations; and Macronix as well as it's suppliers and/or distributors shall be released from any and all liability arisen therefrom.

Copyright© Macronix International Co., Ltd. 2015. All rights reserved, including the trademarks and tradename thereof, such as Macronix, MXIC, MXIC Logo, MX Logo, Integrated Solutions Provider, NBit, Nbit, NBiit, Macronix NBit, eLiteFlash, HybridNVM, HybridFlash, XtraROM, Phines, KH Logo, BE-SONOS, KSMC, Kingtech, MXSMIO, Macronix vEE, Macronix MAP, Rich Au-dio, Rich Book, Rich TV, and FitCAM. The names and brands of third party referred thereto (if any) are for identification purposes only.

For the contact and order information, please visit Macronix's Web site at: http://www.macronix.com